

Machine Learning

Lectures 1 to 4 in One Continuous Read

Every concept, its motivation, its formula, and the derivations the course expects you to be able to do. Based on the lecture notes, slides, exercises, review questions and Tutorial 1 of Lectures 1–4 (Prof. Mounir Ghogho, UM6P).

How to use this document

Read it front to back once. Each part ends with a *thread* box telling you what to carry forward. Formulas are in blue boxes. The reason a concept exists is in yellow. The classic confusions, which the exercises probe directly, are in red. Green *mastery* boxes contain the derivations the professor asks for in the exercises and the tutorial: do not skip them, they are what distinguishes recognising a formula from being able to produce it. Orange boxes work the numbers of exercise-type problems so you can check your own hand calculations. The last pages are a one-glance sheet of every formula and every “*X* versus *Y*” distinction.

Inside the blue boxes, every formula carries its **name** on its left, so you always know what a line is called and can say it out loud. If a symbol stops you, the notation key just below decodes it.

Notation key: how to read the symbols

X, Y / x, y	Capitals are <i>random variables</i> (the input and output before you look); lowercase are particular <i>values</i> .
$x = (x_1, \dots, x_d)^\top$	The input vector; x_j is its j -th feature; d is the number of features; $^\top$ means transpose.
$x^{(i)}, y^{(i)}, x_j^{(i)}$	Superscript (i) = the i -th training example: its input, its label, and feature j of it. N = number of examples.
$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$	The dataset. $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$ are its three subsets.
C, c, k	Number of classes; a class index (c , sometimes k in sums). K = number of folds; k in KNN = number of neighbours.
$P(X, Y),$ $P(Y = c \mid X = x)$	Joint distribution of inputs and outputs; probability that the class is c <i>given</i> the input x (the <i>posterior</i>). $p(y = c \mid x)$ is the same thing.
$p(x \mid y = c), \pi_c$	Class-conditional density (how inputs of class c are distributed); class prior $\pi_c = p(y = c)$ (how common class c is).
$\mathbb{E}[\cdot], \mathbb{E}[\cdot \mid X = x]$	Expectation (average over the distribution); conditional expectation (average over Y for this fixed x).
$\mathbf{1}(A)$	Indicator: 1 if A is true, 0 otherwise.
$\arg \min_a g(a)$	The <i>value of</i> a that makes g smallest (not the smallest value itself). $\arg \max$ likewise.
$\theta, \Theta, f_\theta, \mathcal{H}$	Parameters; the set of allowed parameters; the model with parameters θ ; the hypothesis space $\{f_\theta : \theta \in \Theta\}$.
hat: $\hat{\theta}, \hat{f}, \hat{p}, \hat{\mu}, \hat{R}$	Estimated from data. No hat means the true or population quantity.
star: $f^*, \theta_{\mathcal{H}}^*, t^*$	Optimal: the Bayes rule or best predictor; the best parameters in the family; the best stopping time.
bar: $\bar{f}(x)$	Average over many hypothetical training sets.
$\ell(y, a)$	A loss for one example or one decision. Subscript says which: ℓ_{dec} decision, ℓ_{train} training, ℓ_{01} zero-one, ℓ_{CE} cross-entropy, ℓ_{BCE} binary cross-entropy.
R / $\hat{R}$	<i>Risk</i> = expected loss over the whole distribution (cannot be computed) / <i>empirical risk</i> = average loss over a dataset (computable). Subscript says which dataset: $\hat{R}_{\text{train}}, \hat{R}_{\text{val}}, \hat{R}_{CV}, \hat{R}_{\mathcal{D}}$.
$\hat{p}_\theta(x), \hat{p}_{\theta,c}(x)$	The <i>vector</i> of predicted class probabilities for input x ; its c -th <i>entry</i> . So $\hat{p}_{\theta, y^{(i)}}(x^{(i)})$ = the probability the model gave to the <i>true</i> class of example i .
$s_c(x), \sigma$	Score (logit) of class c ; the sigmoid function.
λ, h, Ω	A hyperparameter; a whole configuration of choices; a penalty function.
$\mu_c, \Sigma_c, \Sigma^{-1}, \Sigma $	Class mean vector; class covariance matrix; its inverse; its determinant.
TP, FP, TN, FN	Attacks flagged; benign flows flagged; benign flows left alone; attacks missed (positive = attack).
$\log$	Natural logarithm (nats) unless written $\log_2$ (bits).

Contents

1	What does it mean to learn? (Lecture 1)	3
1.1	Learning, abilities, and the two ways to give them to a machine	3
1.2	Three forms of experience, three learning settings	4
1.3	Supervised learning in detail	5
1.4	What is presented to the model: features and representations	5
1.5	Four ingredients, and the challenge of generalisation	6
2	From representations to decisions (Lecture 2)	7
2.1	The input as a vector	7
2.2	Decision making when the distribution is known	7

2.3	From the known distribution to finite data	9
2.4	First strategy: estimate locally with nearest neighbours	9
2.5	Second strategy: parameterised models	11
2.6	Learning the parameters: empirical risk minimisation	12
2.7	Evaluating a learned model: three subsets	13
3	From fitting to generalisation (Lecture 3)	14
3.1	Two risks and the generalisation gap	14
3.2	Model capacity, underfitting, overfitting	15
3.3	Bias and variance	16
3.4	Controlling complexity	17
3.5	Feature selection and model selection	18
3.6	Reliable evaluation: splits, metrics, leakage	18
4	Discriminative and generative classification (Lecture 4)	20
4.1	The decision is easy; the posterior is the problem	20
4.2	The discriminative route	20
4.3	The generative route	22
4.4	Gaussian class models: LDA and QDA	22
4.5	Naive Bayes	24
4.6	Comparing the two routes	25
4.7	Imbalance, prior shift, calibration	25
4.8	Unfamiliar inputs, sequences, and combined systems	26
	One-glance sheet: formulas and distinctions	28

1 What does it mean to learn? (Lecture 1)

1.1 Learning, abilities, and the two ways to give them to a machine

Learning is the process through which **experience** changes a learner and develops what it can do. A child learns language, a musician improves with practice, a student becomes able to solve unfamiliar problems. Two things are always present: experience, and a capacity to change in response to it. The lecture's first picture is

$$\text{Capacity to learn} + \text{Experience} \xrightarrow{\text{learning}} \text{Acquired abilities.}$$

For humans the capacity comes from a biological system shaped by evolution; experience acts through observation, practice, interaction, instruction and feedback. The picture is a starting point, since the capacity itself changes with development and prior experience.

The abilities we care about fall into four overlapping activities, which also appear in artificial systems: **recognise** (what is it? objects, people, words, despite variation in appearance), **predict** (what may happen? from experience and current information), **reason** (what follows? working through a new problem rather than recalling an answer), **act** (what should I do? physical and practical skills)

acquired through repeated interaction). Language draws on several at once. The driving question of AI is how to give machines these abilities.

First idea: encode human knowledge. Let humans learn first and transfer the result to the machine as explicit rules: Human knowledge $\rightarrow$ rules or procedures $\rightarrow$ computer. This predates computers (Pascal’s calculator embodied arithmetic procedures in a mechanism) and remains *the right approach* whenever a reliable procedure is known: sorting, tax computation, password checking, a well-defined numerical problem. Learning these from examples would only add uncertainty. Early AI extended the idea to expert domains with **expert systems** (MYCIN): facts and rules obtained from specialists, for instance rules linking symptoms and test results to diagnoses. As systems grew more ambitious, extracting, expressing and maintaining the knowledge became the hard part: the **knowledge acquisition bottleneck**. (Exercise 2 of Chapter 1 asks you to separate three difficulties: obtaining knowledge from the expert, representing it as rules, and facing situations the expert never saw. Keep all three apart.)

Why rules fail for perception. Handwritten digits: people recognise a “3” across changes of position, size, slant and stroke thickness, yet cannot write the procedure they use. Rules about curves, endpoints, dark regions and relative stroke positions meet endless exceptions; every modification to handle one variation (shift, rotation, thickness) introduces new difficulties. Human vision is the broad version: we know much about the visual system and still have no complete explicit description of its computations. The lesson, in one line:

Having an ability $\neq$ having an explicit procedure that describes it.

When useful experience is available but the procedure is hard to specify, a second idea becomes attractive.

Second idea: make the machine learn. Instead of transferring the *result* of human learning, ask whether the machine can acquire the ability from experience itself. The question changes from “what rules should humans write?” to “how can we give a machine the capacity to learn from experience?” Designing the computational methods that make this possible *is* machine learning. It is still programming: the designer builds the learning system and decides what it can represent, what can change through experience, what information guides the change, and what counts as good performance; the task-specific behaviour is then acquired from data instead of being written as an exhaustive rule list.

1.2 Three forms of experience, three learning settings

Experience reaches a machine in three forms, each carrying a different kind of information, and each leads to a setting:

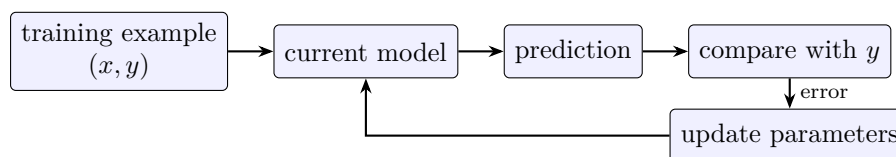
Experience	What it contains	Setting
Examples	an input together with the desired output	supervised learning
Observations	images, text, audio, sensor data with no supplied answer	unsupervised / self-supervised learning
Interactions	actions and their consequences (rewards)	reinforcement learning

Examples give direct guidance about the desired output. Observations contain regularities that can be exploited without labels; self-supervised learning builds its own learning signal from the observations. In interaction the learner partly shapes its future experience, because its actions change what happens next. These are settings, not kinds of systems; one system can use several. The exercise on the hospital’s X-rays (predict pneumonia; group unlabelled images; predict a removed region; train a robot to position a sensor by reward) is exactly this table applied: what experience exists, and what feedback, if any, says the learner is doing well.

1.3 Supervised learning in detail

Each training example pairs an input with a desired output, called the **target**, **label** or **ground truth**. Labels come from measurement (a temperature, a sale price), from human or expert annotation (an object in an image, a reading of a medical scan), or from the future (did the customer repay? did the machine fail within a month?). “Ground truth” deserves care: measurements have errors, experts disagree, labels depend on conventions chosen when the task was defined, and the quality of supervision bounds what can be learned. In the security setting of Tutorial 1 an attack label may come from episode membership even when the individual flow looks benign; the label is then correct *as defined*, and the features simply do not carry enough information to separate it.

The training cycle. The model takes an input and produces a prediction; the prediction is compared with the label; the error guides an update of the parameters; the next example arrives. The computational process through which experience changes the model is **training**.



Regression versus classification. The *output* decides the problem, never the input. Numerical output (price, temperature, remaining lifetime, rainfall amount) is **regression**; categorical output (digit identity, spam or not, benign or malignant, species) is **classification**. The same medical image gives regression (tumour size) or classification (tumour type) depending on the question; the same weather data predicts rainfall amount or whether it rains. Richer outputs exist: object detection (several objects and locations), semantic segmentation (a class per pixel). The student-records exercise is the test: predicting graduation is classification, predicting the final grade is regression, grouping trajectories with no target is neither (unsupervised).

Prediction, decision, action. A **prediction** is what the model says about the unknown output. A **decision** selects a response using the prediction and a criterion. An **action** is what is executed. A medical model predicts a disease probability; the decision is whether to order a test; the action is the test. They coincide in simple cases and separate exactly when different errors have different consequences, which Lecture 2 makes precise. The tutorial asks why a posterior estimate and an alert are not the same object: the posterior is a number summarising evidence; the alert is a choice that also depends on the costs of false alarms and missed attacks.

1.4 What is presented to the model: features and representations

The form of experience says what information exists; a separate question is what part is *presented* to the model. For a house-price model a designer chooses features (area, location, age, rooms) and the model learns how they relate to price; some supplied features may turn out to matter little. For perception this choice is far harder: an image is millions of pixels, audio is a long waveform, text meaning depends on relations among many words. For years systems relied on **feature engineering**: edges, corners, textures and shapes for vision; spectral features and mel-frequency cepstral coefficients for speech. Domain knowledge decided what to present; the model learned how to use it. The representation has a major effect on success.

Representation learning lets the machine discover a useful representation from experience. The idea predates deep learning, but deep learning suits it because successive layers build higher-level features (parts, objects, abstract scene properties) from lower-level ones (local patterns):

raw input $\rightarrow$ human-designed features $\rightarrow$ learned predictor
 versus raw input $\rightarrow$ learned representation $\rightarrow$ learned predictor.

Humans have not disappeared: they still choose architecture, learning method, experience and objective. What changed is that more of the representation is acquired from data.

Self-supervision. A learning problem can be built from observations alone: predict a missing word from its context, a missing image region from the rest, a missing audio segment. The task gives the model a reason to discover regularities, so large unlabelled collections become useful; the representation can then be transferred to tasks with few labels (an image model adapted to classification or detection, a language model reused for several language tasks).

From self-supervision to generation. Text supplies both the context and what comes next, so an **autoregressive language model** trained to predict the next token is self-supervised. Once trained, generation is repeated prediction: predict a token, append it, predict again. Two uses of one model, not two kinds of model: learned representations for downstream tasks, and learned predictive ability for generation.

1.5 Four ingredients, and the challenge of generalisation

The four ingredients of any learning system

Experience	what information is available?
Model	how can acquired knowledge be represented?
Learning algorithm	how does experience change the model?
Evaluation criterion	how is success judged?

The model determines what can be represented, the algorithm how experience changes it, the criterion which changes are useful, and the experience what evidence exists at all. Representation learning adds that the internal representation itself can be shaped by experience. The exercise “design a system without naming an algorithm” is this box: experience available, form of the output, information the algorithm can use, how success is judged.

Generalisation. Experience is finite; deployment is not. **Generalisation** is the ability to perform well in situations not encountered during learning. A recogniser perfect on its own examples may fail on a new handwriting style, so performance on seen examples is only partial evidence about what was learned:

$$\text{Finite experience} \xrightarrow{\text{learning}} \text{ability useful in new situations.}$$

Three exercise scenarios make the point concrete. *Wolves and snow*: a wolf-versus-dog system succeeds because wolves were photographed on snow, then fails after deployment; a wolf on grass and a dog on snow reveal which cue it used. *Two bird collections*: a million photos from one camera in one park versus fifty thousand from many people, countries, seasons and angles; amount of experience is not the same as its diversity and representativeness. *The simulated warehouse*: a robot trained on flat floors, constant friction and fixed lighting fails in reality; varying those conditions matters more than visual realism. And the advertising claim “our AI has learned every possible situation” is incoherent: the situations met after deployment are not the ones observed.

Connection to information theory. Entropy measures uncertainty; information measures how much observing something reduces it. An input is useful for prediction when it carries information about the output. Two links to keep: **cross-entropy** measures how good predicted probabilities are and becomes the standard classification training criterion (Lecture 2); representation learning asks which information a representation should preserve.

The thread so far

Learning = experience changing a model so that it works on new situations. Supervised learning uses labelled examples; the output type decides regression versus classification; prediction, decision and action are distinct; the representation presented to the model matters and may itself be learned; everything is judged by generalisation, never by fit to the examples already seen.

2 From representations to decisions (Lecture 2)

2.1 The input as a vector

The learner never receives the house or the image; it receives selected properties, encoded as numbers:

$$\text{object} \rightarrow \text{features} \rightarrow x = [x_1, \dots, x_d]^\top \in \mathcal{X} \subseteq \mathbb{R}^d.$$

A house with area 145, four rooms, age 12 and neighbourhood A (one of three) becomes the vector $x = (145, 4, 12, 1, 0, 0)^\top$. Numerical components are copied, the categorical one encoded by three binary components (**one-hot**). Components can live in $[0, 1]$, in $\{0, 1\}$, or in $\mathbb{R}$. The representation fixes what information the learner can use: area alone says less about price than area, location, age, condition and rooms. More relevant components help; irrelevant, redundant or unreliable ones raise d without adding information. Pixels and waveforms are numerical but do not encode shape or meaning explicitly, which is why representation learning matters.

Mastery: representation creates unavoidable ambiguity (Exercise 1, Ch. 2)

Suppose two physically different events z_A (benign) and z_B (malicious) map to the *same* vector $x = g(z_A) = g(z_B)$. A deterministic classifier is a function of x alone, so it outputs one label for both events and is wrong on one of them: no classifier using only x can be right on both. If among events with this x 30% are malicious, the Bayes decision under equal costs is “benign” (the more probable class) with conditional error 0.30; this error is in the *problem*, not in the labels, which may be perfectly correct. A new feature that separates the two is legitimate if it is available at decision time (e.g. the destination port) and leakage if it is only known afterwards (e.g. an analyst’s incident ticket). Adding *irrelevant* features leaves the Bayes error unchanged (the conditional distribution of Y given the informative part is the same) but makes learning from finite data harder: neighbourhoods spread out and more parameters must be estimated.

2.2 Decision making when the distribution is known

Idealise: the input is a random vector X , the output a random variable Y , with joint distribution $P(X, Y)$ describing which pairs occur and how often. Once $X = x$ is observed, what remains is the **conditional distribution** $P(Y | X = x)$: houses with identical descriptions still sell at different prices because information is missing, measurements are imperfect, markets vary. The conditional distribution carries that leftover uncertainty instead of forcing one output per input.

Knowing probabilities does not say what to do; consequences do. If the true output is y and we choose an action $a \in \mathcal{A}$, the **decision loss** $\ell_{\text{dec}}(y, a)$ is the cost. The action space is a set of numbers in regression, the set of labels in classification, and may differ from the set of outcomes in general (“request human review” is an action, not an outcome).

Conditional risk and the Bayes decision rule

decision loss $\ell_{\text{dec}}(y, a) = \text{cost of taking action } a \text{ when the truth is } y$

conditional risk of action a $\mathbb{E}[\ell_{\text{dec}}(Y, a) | X = x]$

Bayes decision rule $f^*(x) \in \arg \min_{a \in \mathcal{A}} \mathbb{E}[\ell_{\text{dec}}(Y, a) | X = x]$

In words: the conditional risk is the average cost you expect if you take action a for this input, averaging over the outputs that are still possible; the Bayes rule takes the action with the smallest such average. “Bayes” means optimal decision under a probability distribution; no prior on parameters is involved.

Regression: the loss decides which summary to report.

Mastery: squared, absolute and asymmetric loss

Squared loss. For fixed x write $m = \mathbb{E}[Y | x]$. Then $\mathbb{E}[(Y - a)^2 | x] = \mathbb{E}[(Y - m)^2 | x] + (m - a)^2$ (expand $(Y - a) = (Y - m) + (m - a)$; the cross term is $2(m - a)\mathbb{E}[Y - m | x] = 0$). The first term does not depend on a , so the minimiser is $a = m$: **the conditional mean** $f^*(x) = \mathbb{E}[Y | X = x]$.

Absolute loss. $\frac{d}{da} \mathbb{E}|Y - a| = P(Y < a) - P(Y > a)$, zero when $P(Y \leq a) = \frac{1}{2}$: any **conditional median**.

Asymmetric loss $\ell(y, a) = c_u(y - a)$ if $a < y$ (under-prediction) and $c_o(a - y)$ if $a \geq y$ (over-prediction). Differentiating the expected loss, $c_o F(a) - c_u(1 - F(a)) = 0$, so the optimum is the **conditional quantile** at level

$$F(a^*) = \frac{c_u}{c_u + c_o}.$$

When under-prediction is costlier ($c_u > c_o$) the quantile is above the median: predict high to avoid the expensive error.

Worked numbers: exponential response time (Exercise 2, Ch. 2)

$p(y | x) = \frac{1}{4}e^{-y/4}$, $F(y) = 1 - e^{-y/4}$. Squared loss: mean = 4. Absolute loss: median solves $1 - e^{-a/4} = \frac{1}{2}$, $a = 4 \ln 2 \approx 2.77$. Asymmetric loss with $c_u = 4$, $c_o = 1$: quantile level $4/5$, $1 - e^{-a/4} = 0.8$, $a = 4 \ln 5 \approx 6.44$. Three different “best predictions” for one and the same $P(Y | x)$. A model that outputs only the conditional mean cannot serve the asymmetric loss later; estimating the whole conditional distribution (or several quantiles) can.

So “the best prediction” has no meaning until the loss is specified.

Classification. With C classes the conditional distribution is the vector of posteriors $P(Y = c | X = x)$, $c = 1, \dots, C$. Prediction (probabilities) and decision (a class) are visibly separate: $\{P(Y = c | x)\}_c \xrightarrow{\text{decision criterion}} f^*(x)$.

Bayes classifier under zero-one loss

zero-one loss $\ell_{01}(y, a) = \mathbf{1}(y \neq a)$

its conditional risk $\mathbb{E}[\ell_{01}(Y, a) | X = x] = 1 - P(Y = a | X = x)$

Bayes classifier $f^*(x) = \arg \max_c P(Y = c | X = x)$

In words: every mistake costs 1 and a correct answer costs 0, so the expected cost of answering a is the probability that a is wrong; minimising it means choosing the most probable class.

Mastery: the cost threshold, and the reject option (Tutorial Q2, Exercise 3)

Binary case, class 1 = the event to detect, $p = P(Y = 1 | x)$, false-positive cost C_{FP} , false-negative cost C_{FN} , correct decisions free. Conditional risks:

$$R(\text{declare } 1 | x) = C_{FP}(1 - p), \quad R(\text{declare } 0 | x) = C_{FN}p.$$

Declare 1 when $C_{FP}(1 - p) < C_{FN}p$, i.e. $C_{FP} < (C_{FP} + C_{FN})p$:

$$\text{decide positive} \iff p > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Equal costs give 0.5. A costlier false negative *lowers* the threshold (less evidence needed to alarm); a costlier false positive raises it.

Three actions. Add “request human review” at fixed cost C_R , after which the case is resolved correctly: $R(\text{review} | x) = C_R$, a horizontal line. Benign is optimal while $C_{FN}p \leq \min(C_R, C_{FP}(1 - p))$, attack while $C_{FP}(1 - p) \leq \min(C_R, C_{FN}p)$, review in between. With C_R

small enough to matter, the regions are

$$p \leq \frac{C_R}{C_{FN}} : \text{benign}, \quad \frac{C_R}{C_{FN}} < p < 1 - \frac{C_R}{C_{FP}} : \text{review}, \quad p \geq 1 - \frac{C_R}{C_{FP}} : \text{attack}.$$

One threshold no longer suffices because the cheapest action changes twice. Raising C_R shrinks the review interval until it vanishes and the single threshold returns.

Worked numbers: Tutorial Q2

$C_{FP} = 2$, $C_{FN} = 8$: threshold $2/10 = 0.2$. Posteriors 0.08, 0.35, 0.85 give benign, attack, attack. With review at 0.9: benign for $p \leq 0.9/8 = 0.1125$, attack for $p \geq 1 - 0.9/2 = 0.55$, review for $0.1125 < p < 0.55$ (check at $p = 0.3$: benign 2.4, attack 1.4, review 0.9). Exercise 3 of Chapter 2 with costs 1, 9 and review 0.6: benign for $p \leq 0.067$, attack for $p \geq 0.4$, review between.

The probabilities say what the input tells you; the loss says how to use it. The same $p(x)$ leads to different decisions in different applications. This is also why changing the threshold to reflect costs is *not* a correction for class imbalance: the threshold acts on a fixed, correct posterior; imbalance is about whether that posterior was estimated well (Lecture 4).

2.3 From the known distribution to finite data

In reality P is unknown. What we have is a finite **training set**

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N, \quad x_j^{(i)} = \text{feature } j \text{ of example } i,$$

modelled as independent, identically distributed (**i.i.d.**) draws from $P(X, Y)$. This is a starting point: time series, spatial measurements and interacting agents violate independence, and the deployment distribution may differ when the population or the measurement process changes. The central statistical step of supervised learning: use the finite sample to build a rule that approximates the Bayes rule we would use if P were known.

2.4 First strategy: estimate locally with nearest neighbours

The Bayes rules need the conditional mean or the conditional class probabilities at x . If many examples had exactly input x we would average their outputs; exact repeats are rare with continuous features, so use the examples *near* x . Let $N_k(x)$ be the indices of the k training inputs closest to x .

k -nearest neighbours

neighbourhood of x $N_k(x)$ = indices of the k training inputs closest to x

KNN regression estimate $\hat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}$

KNN class-probability estimate $\hat{P}_k(Y = c | x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c)$

KNN classifier (majority vote) $\hat{y}(x) = \arg \max_c \hat{P}_k(Y = c | x)$

In words: average the neighbours' outputs; the estimated probability of class c is the fraction of neighbours that have class c ; under zero-one loss pick the class most neighbours have. With $k = 1$ the decision boundary between two training points is the perpendicular bisector of the segment joining them (the points closer to A than to B).

“Nearest” depends on the distance and the representation. The Minkowski family

$$d_p(x, z) = \left(\sum_{j=1}^d |x_j - z_j|^p \right)^{1/p}, \quad p = 1 \text{ Manhattan, } p = 2 \text{ Euclidean, Hamming for binary vectors}$$

($x = (0, 0)$, $z = (2, 3)$): $d_1 = 5$, $d_2 = \sqrt{13}$, and a weighted version $d_w(x, z) = \sqrt{\sum_j w_j (x_j - z_j)^2}$.

Scaling matters: standardise each feature, $z_j = (x_j - \mu_j)/\sigma_j$, or the feature with the largest units decides everything. Irrelevant features make similar examples look far apart. One-hot encoding plus Euclidean distance silently assumes every pair of distinct categories is equally far apart; a domain-aware distance (a similarity table between locations) is the alternative.

Worked numbers: KNN and units (Exercise 5, Ch. 2; Tutorial Q3)

Query (50,000, 30), A = (51,000, 60), B = (60,000, 31). Raw: $d_A = \sqrt{1000^2 + 30^2} \approx 1000.4$, $d_B = \sqrt{10000^2 + 1} \approx 10000$: A is nearest, a person thirty years older. Scale income by 10,000 and age by 10: query (5, 3), A (5.1, 6), B (6, 3.1); $d_A = \sqrt{0.01 + 9} \approx 3.0$, $d_B = \sqrt{1 + 0.01} \approx 1.0$: B is nearest. Nothing changed about the people; only the geometry imposed on them.

Tutorial: query (50, 10), A = (60, 100) attack, B = (400, 12) benign. Raw: $d_A = \sqrt{100 + 8100} \approx 90.6$, $d_B = \sqrt{350^2 + 4} \approx 350$: A. Scaled by (100, 20): $z = (0.5, 0.5)$, $z_A = (0.6, 5)$, $z_B = (4, 0.6)$; $d_A = \sqrt{0.01 + 20.25} \approx 4.50$, $d_B = \sqrt{12.25 + 0.01} \approx 3.50$: B, predicted benign. Weighted $d_w^2 = w(z_1 - z_1')^2 + (z_2 - z_2')^2$: A is nearer when $0.01w + 20.25 < 12.25w + 0.01$, i.e. $w > 20.24/12.24 \approx 1.65$. Below the critical weight B wins, above it A wins; w is a modelling choice about feature importance and must be chosen on validation data, not by looking at the test answers.

The hyperparameter k . Small k follows individual observations (sensitive to noise, jagged boundary); large k averages broadly (smoother, can erase real local structure). It is not estimated by the averaging rule; it is chosen during development: a **hyperparameter**. Raising k reduces capacity (Lecture 3).

Why this exists

KNN is the most direct translation of the Bayes rule into data. Its weakness in high dimension is what motivates everything after it.

Mastery: the curse of dimensionality, two ways

Grid count. On $[0, 1]^d$ a grid with ten positions per axis has 10^d cells: 100 for $d = 2$, 10^{10} for $d = 10$. A fixed dataset becomes sparse; a neighbourhood holding k points must cover a large region and the estimate loses its locality.

Binomial calculation (Exercise 6, Ch. 2). n points uniform in the unit d -ball, query at the centre, useful neighbour within radius r . One point lands inside with probability equal to the volume ratio r^d , so $M_d \sim \text{Binomial}(n, r^d)$, $\mathbb{E}M_d = nr^d$, $P(M_d \geq 1) = 1 - (1 - r^d)^n$. With $n = 1000$, $r = 0.1$: $d = 3$ gives $1 - 0.999^{1000} \approx 0.63$; $d = 4$ gives $1 - (1 - 10^{-4})^{1000} \approx 0.095 < 0.5$. The first failing dimension is 4. For $P(M_d \geq 1) = 0.95$ one needs $r_d = (1 - 0.05^{1/n})^{1/d} \approx (3/n)^{1/d} \rightarrow 1$: the “neighbourhood” grows until it is the whole ball. The calculation is even generous to KNN: the query sits at the centre, the data are uniform, and no dimension is irrelevant.

The curse does not make every high-dimensional problem hopeless; it shows that local estimation cannot exploit high-dimensional inputs without extra structure. Hence the second strategy: describe how predictions vary across the whole input space with a shared model and learn its adjustable quantities from *all* examples.

2.5 Second strategy: parameterised models

A **parameterised model** is a function $f_\theta(x)$ whose adjustable quantities θ are learned from data. The set it makes available is the **hypothesis space**

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}.$$

The model *form* decides how parameters are shared across inputs; θ picks one member. Choosing $\mathcal{H}$ introduces structure before any learning.

Regression models. Linear regression $f_\theta(x) = w^\top x + b$, $\theta = (w, b)$: changing w, b moves within the family. A polynomial $f_\theta(x) = \theta_0 + \theta_1 x + \dots + \theta_p x^p$ shows two levels: the coefficients are learned parameters, the degree p specifies the family.

Classification models. The model outputs an estimated probability vector in the **probability simplex**

$$\hat{p}_\theta(x) = [\hat{P}_\theta(Y = 1 | x), \dots, \hat{P}_\theta(Y = C | x)]^\top \in \Delta^{C-1} = \left\{p \in \mathbb{R}^C : p_c \geq 0, \sum_c p_c = 1\right\},$$

and the decision rule (largest probability, cost threshold, reject option) is applied afterwards to that vector.

Cross-entropy loss (also called log loss)

predicted probability vector $\hat{p}_\theta(x^{(i)}) = (\hat{p}_{\theta,1}(x^{(i)}), \dots, \hat{p}_{\theta,C}(x^{(i)}))$

probability of the true class $\hat{p}_{\theta,y^{(i)}}(x^{(i)})$

per-example cross-entropy $\ell_{CE}(y^{(i)}, \hat{p}_\theta(x^{(i)})) = -\log \hat{p}_{\theta,y^{(i)}}(x^{(i)})$

empirical cross-entropy (training loss) $\hat{R}_{CE}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{\theta,y^{(i)}}(x^{(i)})$

binary cross-entropy, $y \in \{0, 1\}$ $\ell_{BCE}(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$

In words: the model outputs one probability per class; take the one belonging to the correct label; the loss of the example is minus its log; the training loss is the average of that over the N examples. The binary form is the same thing for two classes, with $\hat{p}$ the probability of class 1: if $y = 1$ only $-\log \hat{p}$ survives, if $y = 0$ only $-\log(1 - \hat{p})$. Example: three classes, prediction $(0.7, 0.2, 0.1)$; true class 1 gives $-\ln 0.7 = 0.36$, true class 3 gives $-\ln 0.1 = 2.30$.

Assigning 0.9, 0.6, 0.5, 0.1, 0.01 to the true class costs 0.105, 0.511, 0.693, 2.30, 4.61: graded, and confident mistakes are punished hard. Zero-one loss cannot play this role: it is the same for a confident and a hesitant correct answer, changes only when the predicted class flips, and (slides) is piecewise constant, so its gradient is zero or undefined and nothing can be optimised. Cross-entropy is the smooth **surrogate** that rewards better probability estimates.

Worked numbers: training loss and decision performance disagree (Exercise 4, Ch. 2)

Four examples; probability given to the correct class. Model A: 0.6, 0.6, 0.6, 0.6. Model B: 0.99, 0.99, 0.99, 0.20. Zero-one at threshold 0.5: A makes 0 errors, B makes 1 (0.25). Average cross-entropy: A = $-\ln 0.6 = 0.511$; B = $(3 \times 0.010 + 1.609)/4 = 0.410$. The two criteria rank the models oppositely. This does not make cross-entropy a bad training loss: it is estimating probabilities, and B's probabilities are more informative on three cases and honestly uncertain on none. The decision loss is applied afterwards; a training loss and a decision loss answer different questions.

Mastery: why cross-entropy learns the true probabilities

For a true distribution p and an estimate $\hat{p}$ on the same alphabet,

$$\text{entropy} \quad H(p) = - \sum_c p_c \log p_c$$

$$\text{cross-entropy} \quad H(p, \hat{p}) = - \sum_c p_c \log \hat{p}_c$$

$$\text{Kullback–Leibler divergence} \quad D_{KL}(p \parallel \hat{p}) = \sum_c p_c \log \frac{p_c}{\hat{p}_c}$$

Expanding the divergence, $D_{KL} = \sum_c p_c \log p_c - \sum_c p_c \log \hat{p}_c = -H(p) + H(p, \hat{p})$, hence

$$H(p, \hat{p}) = H(p) + D_{KL}(p \parallel \hat{p}).$$

$H(p)$ is fixed by the source and $D_{KL} \geq 0$ with equality iff $\hat{p} = p$, so expected cross-entropy is smallest exactly at the true distribution. Apply this at every input with $p_c(x) = P(Y = c \mid x)$:

$$\text{expected cross-entropy risk} \quad R_{CE}(\theta) = H(Y \mid X) + \mathbb{E}_X [D_{KL}(p(X) \parallel \hat{p}_\theta(X))]$$

$$\text{mutual information} \quad I(X; Y) = H(Y) - H(Y \mid X)$$

The learner shrinks the second term by moving its probabilities toward the truth; the first term, the uncertainty left after seeing X , belongs to the problem, so no classifier beats $H(Y \mid X)$. An informative representation has larger mutual information with the class. Coding view: probability $\hat{p}_c$ is a code of $-\log_2 \hat{p}_c$ bits, the excess over $H_2(p)$ is D_{KL} , and a one-hot target reduces the per-example cross-entropy to $-\log \hat{p}_{\theta, y^{(i)}}$. Minimising cross-entropy also maximises the conditional likelihood (Lecture 4).

Parameters versus hyperparameters. Parameters are learned from training examples (polynomial coefficients, network weights). Hyperparameters are choices around that estimation (degree, k , number of layers, regularisation strength): some define the hypothesis space, others control the learning procedure.

model form and hyperparameters $\rightarrow \mathcal{H} \rightarrow \theta \rightarrow$ particular model.

The labels are not intrinsic (Exercise 8): one system picks the degree p by validation and fits $p + 1$ coefficients; another always uses degree 20 with an L_1 penalty that zeroes most coefficients. Both can end with the same effective degree, reached by different procedures; in the second, the degree is fixed and the penalty strength is the hyperparameter. Comparing the two fairly needs data kept outside both procedures' parameter fitting.

2.6 Learning the parameters: empirical risk minimisation

Two losses must be told apart: ℓ_{train} guides estimation from data; ℓ_{dec} guides the action. They may coincide (squared loss for both) or differ (train with cross-entropy, decide with zero-one or application costs).

Population risk, empirical risk, ERM

$$\begin{aligned}
\text{population risk (true risk)} \quad R(\theta) &= \mathbb{E}_{(X,Y) \sim P} [\ell_{\text{train}}(Y, f_{\theta}(X))] \\
\text{best parameters in the family} \quad \theta_{\mathcal{H}}^* &\in \arg \min_{\theta \in \Theta} R(\theta) \\
\text{empirical risk (training risk)} \quad \hat{R}_{\text{train}}(\theta) &= \frac{1}{N} \sum_{i=1}^N \ell_{\text{train}}(y^{(i)}, f_{\theta}(x^{(i)})) \\
\text{ERM estimate} \quad \hat{\theta} &\in \arg \min_{\theta \in \Theta} \hat{R}_{\text{train}}(\theta)
\end{aligned}$$

In words: the population risk is the average loss over every case the problem can produce (unknown); the empirical risk is the average loss over the training examples (computable); ERM picks the parameters that make the training average smallest.

Population risk averages the loss over all pairs the problem can produce, including unseen ones; it is what we care about and cannot be computed. The empirical risk replaces the expectation by the sample average, and choosing θ to minimise it is **empirical risk minimisation (ERM)**: every candidate is judged on the same observations. $\theta_{\mathcal{H}}^*$ may be worse than the Bayes rule if $\mathcal{H}$ does not contain it. The hat on $\hat{\theta}$ records dependence on the particular sample; another sample gives another $\hat{\theta}$.

Mastery: ERM alone cannot choose (Exercises 7 and 9, Ch. 2)

Data $(-1, 1), (0, 0), (1, 1)$. $f_1(x) = x^2$ fits exactly. So does $f_{2,\alpha}(x) = x^2 + \alpha(x+1)x(x-1)$ for every α , because the extra factor vanishes at $-1, 0, 1$. Zero training error for all of them; at $x = \frac{1}{2}$ they predict $\frac{1}{4} - \frac{3}{8}\alpha$, anything at all. ERM is silent among them. Preferring the quadratic is an assumption about the hypothesis space; regularisation is an assumption that small coefficients are more plausible; validation data is new evidence. Same lesson with two classifiers that agree on regions A, B and differ on region C that the sample never visited: training loss ties, population risk may differ either way (make C mostly class 1 or mostly class 0), the tie is broken by the algorithm's implicit preference, and only new data from C, domain knowledge, or a deliberate collection effort gives evidence.

Do not confuse

ERM is a good way to estimate *parameters* inside a fixed configuration, and a bad way to choose *complexity*. If $\mathcal{H}_1 \subseteq \mathcal{H}_2$, the best training fit in $\mathcal{H}_2$ can only be as good or better, and part of that improvement is fitting accidental variation. Raising the degree or lowering k keeps reducing training loss after performance on new data has started to degrade. Parameters need training data; hyperparameters need separate validation data.

Objective versus algorithm. The learning objective says which parameter values are preferred (ERM). The learning algorithm says how they are found: sometimes a closed form, usually iterative optimisation (later lectures). The full sequence separates three stages:

$$\mathcal{D} \xrightarrow[\text{learning}]{\ell_{\text{train}}} \text{learned model} \xrightarrow[\text{inference}]{x} \text{inferred quantity} \xrightarrow[\text{decision}]{\ell_{\text{dec}}} \text{action}.$$

2.7 Evaluating a learned model: three subsets

The population risk is unavailable, so held-out data estimate behaviour beyond the sample. The three subsets differ by what each is allowed to influence:

Training set $\mathcal{D}_{\text{train}}$	learn the parameters (and every data-dependent preprocessing step)
Validation set $\mathcal{D}_{\text{val}}$	compare fitted configurations; select k , degree, features, thresholds, model family
Test set $\mathcal{D}_{\text{test}}$	one final estimate after everything is frozen

A subset stays informative only while its outputs are not used to make an earlier choice. Reusing validation results for many decisions gradually adapts the development to that set, hence the third set; consulting the test set during development turns it into another validation set and makes its estimate optimistic.

Hyperparameter selection and K -fold cross-validation

For each candidate λ , learn parameters on training data, then compare the fitted models on validation data. When data are limited, split the development data into K folds, each serving once as the validation set:

$$\text{parameters fitted for a given } \lambda \quad \hat{\theta}_\lambda = \arg \min_{\theta} \hat{R}_{\text{train}}(\theta; \lambda)$$

$$\text{selected hyperparameter} \quad \hat{\lambda} = \arg \min_{\lambda} \hat{R}_{\text{val}}(\hat{\theta}_\lambda; \lambda)$$

$$K\text{-fold cross-validation risk} \quad \hat{R}_{CV}(\lambda) = \frac{1}{K} \sum_{j=1}^K \hat{R}_{\text{val}}^{(j)}(\lambda)$$

$\hat{R}_{\text{val}}^{(j)}(\lambda)$ is the validation risk on fold j of the model trained on the other $K - 1$ folds. Every example validates once and trains $K - 1$ times; after choosing $\hat{\lambda}$, refit on all development data; the test set stays untouched.

The validation criterion need not equal the training loss; it should reflect what the application values.

The thread so far

Known P + a loss $\Rightarrow$ Bayes rule: conditional mean / median / quantile in regression, most probable class or the cost threshold $C_{FP}/(C_{FP} + C_{FN})$ in classification, with a review band when a third action exists. Unknown $P \Rightarrow$ finite i.i.d. sample. Estimate locally (KNN; geometry of the representation; killed by dimension) or through a parameterised family fitted by ERM, with cross-entropy as the probability-learning loss. Training data fit θ , validation data choose hyperparameters, the test set is spent once.

3 From fitting to generalisation (Lecture 3)

3.1 Two risks and the generalisation gap

ERM solves a *fitting* problem. The purpose of learning is performance on new cases, so compare two averages of the same loss for a fitted model $\hat{f}_{\mathcal{D}}$:

Empirical risk, population risk, generalisation gap

$$\text{empirical risk of } f \text{ on } \mathcal{D} \quad \hat{R}_{\mathcal{D}}(f) = \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, f(x^{(i)}))$$

$$\text{population risk of } f \quad R(f) = \mathbb{E}_{(X,Y) \sim P}[\ell(Y, f(X))]$$

$$\text{generalisation gap} \quad G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$$

In words: the gap is how much worse the fitted model does on new cases than on the cases it was fitted to. Only the empirical risk is observable; the population risk needs the unknown P , and the gap needs both.

Both terms concern the same model; only the source of examples differs. A small gap means training performance transfers; a large positive gap means the training number flatters. Even with i.i.d. draws the finite average does not equal the expectation.

Why training performance is special. Training data are used twice: to select the model and then to evaluate it, $\mathcal{D} \rightarrow \hat{f}_{\mathcal{D}} \rightarrow \hat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$, so the procedure can exploit accidental regularities of $\mathcal{D}$. Another sample $\mathcal{D}'$ would give another $\hat{f}_{\mathcal{D}'}$; this sampling variability is part of the problem, not a defect of notation. Low training loss is necessary evidence that optimisation worked and incomplete evidence that learning succeeded.

Stable versus spurious regularities. Cows photographed on grass, camels on sand: shape and background both predict every training label. Shape is the intended cue; background is a **shortcut** created by data collection. A cow on sand tells them apart; training performance cannot. Dataset composition is therefore part of the method. In security the same happens when a network segment, a capture time or a device identifier predicts attacks: predictive in the collected data, useless at deployment. The tutorial’s night-time attacks are a shortcut, and it is *different from leakage*: time of day *is* available at decision time, it is just not a stable cause; leakage uses information that would not be available at all. Vary nuisance factors when collecting; use **data augmentation** only with realistic, label-preserving transformations (an unrealistic one changes the problem).

Inductive bias. A finite sample is compatible with many functions, including ones identical on the sample and different everywhere else (Exercise 7 above). Learning therefore needs assumptions that make some solutions more plausible than others. They enter through four doors:

- the data and representation decide which cues are available;
- the hypothesis space decides which relationships can be expressed;
- regularisation favours some fitting solutions over others;
- the optimisation algorithm may prefer particular solutions even when many minimise the objective.

“Bias” here means preference, broader than the statistical bias below. Generalisation succeeds when these assumptions stay valid at deployment.

3.2 Model capacity, underfitting, overfitting

Capacity is how varied the functions in $\mathcal{H}$ can be. It is not the parameter count alone: structure, constraints, regularisation, optimisation and the data’s amount and geometry all enter. Capacity-controlling hyperparameters: polynomial degree, tree depth, number of units, and k in KNN, where *smaller* k means more capacity because the fitted function can change at every point.

Mastery: nested families and the monotone training error (Review Q5, Tutorial Q5a)

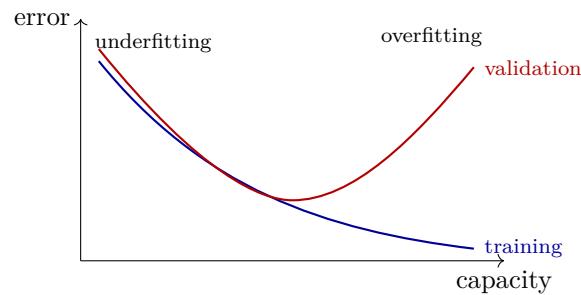
For nested families $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \dots$, every function in $\mathcal{H}_m$ is also in $\mathcal{H}_{m+1}$, so the minimum over the larger set is taken over a superset and cannot be larger:

$$\min_{f \in \mathcal{H}_{m+1}} \hat{R}_{\text{train}}(f) \leq \min_{f \in \mathcal{H}_m} \hat{R}_{\text{train}}(f).$$

That is the whole proof. It explains why training error alone always favours the richer family, and says nothing about population risk: the extra functions can fit stable structure or accidental variation.

Underfitting: too restrictive, high training *and* validation error (a line through curved data).

Overfitting: too sensitive to the particular sample, low training error and much higher validation error (a high-degree polynomial oscillating between points). The question is not whether the model is complicated in isolation but whether its effective flexibility fits the available evidence; a family unstable on a small sample can be well constrained by a large one.



Worked numbers: Tutorial Q5

Three nested families: training errors 0.20, 0.08, 0.00, validation errors 0.22, 0.11, 0.19. Select $\mathcal{H}_2$. $\mathcal{H}_1$ underfits (both errors high, close together). $\mathcal{H}_3$ interpolates the sample and shows a large gap: estimation variance from excessive capacity for this sample size. After collecting much more data $\mathcal{H}_3$'s validation error drops to 0.10: the same family is now well constrained. Capacity is judged relative to N .

3.3 Bias and variance

Imagine redrawing training sets $\mathcal{D}_1, \mathcal{D}_2, \dots$ from the same population and running the same procedure; the fitted functions differ. For squared-error regression let $f^*(x) = \mathbb{E}[Y \mid X = x]$ and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$, the average fitted prediction at x .

Bias and variance at a fixed input

optimal predictor (squared loss) $f^*(x) = \mathbb{E}[Y \mid X = x]$

average fitted prediction $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$

bias at x $\text{Bias}(x) = \bar{f}(x) - f^*(x)$

variance at x $\text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2]$

$\mathbb{E}_{\mathcal{D}}$ averages over hypothetical training sets drawn from the same population and fed to the same procedure.

Bias: would the *average* fit miss the target in a consistent direction? Restrictive models and poor features cause it, and more data does not remove it if the family cannot express the relationship. **Variance:** does swapping the sample change the prediction at the same input? High capacity, small data, irrelevant variables and unstable feature selection raise it.

Mastery: the bias–variance decomposition

Assume $Y = f^*(X) + \varepsilon$ with $\mathbb{E}[\varepsilon \mid X] = 0$ and $\text{Var}(\varepsilon \mid X = x) = \sigma^2(x)$, the noise independent of the training sample. At fixed x write

$$Y - \hat{f}_{\mathcal{D}}(x) = \underbrace{(Y - f^*(x))}_{\varepsilon} + \underbrace{(f^*(x) - \bar{f}(x))}_{-\text{Bias}(x)} + \underbrace{(\bar{f}(x) - \hat{f}_{\mathcal{D}}(x))}_{\text{deviation}}.$$

Square and take the expectation over ε and $\mathcal{D}$. The three cross terms vanish: $\mathbb{E}[\varepsilon] \cdot (\dots) = 0$; $\mathbb{E}[\varepsilon(\bar{f} - \hat{f}_{\mathcal{D}})] = 0$ because ε and $\mathcal{D}$ are independent; $(f^* - \bar{f}) \mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}_{\mathcal{D}}] = 0$ by definition of $\bar{f}$. What remains is

$$\mathbb{E}_{\mathcal{D}, Y|x}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \underbrace{\sigma^2(x)}_{\text{irreducible noise}} + \underbrace{\text{Bias}(x)^2}_{\text{systematic error}} + \underbrace{\text{Var}(x)}_{\text{sample sensitivity}}.$$

The noise term stays even for the optimal predictor f^* : it is the part of Y that X does not determine.

Raising capacity typically lowers bias and raises variance, producing the U-shaped validation curve; this is a diagnostic principle, not a universal law.

Beyond the U. Overparameterised models can show **double descent**: validation error spikes near the **interpolation threshold** (the smallest capacity at which the training sample can be fitted exactly) and may fall again beyond it, depending on regularisation, the optimiser, data geometry and which interpolating solution is selected. The same can happen over *training time* at fixed size. So examine validation behaviour rather than assuming the U. Three sources of excess error: **approximation error** ($\mathcal{H}$ cannot express a good function), **estimation error** (finite data cannot identify the best member), **optimisation error** (the algorithm fails to reach a good solution of its objective). More capacity trades approximation error for estimation error.

3.4 Controlling complexity

Options: restrict the family (depth, degree), raise k , select features, penalise parameters, stop early, collect more relevant data. Each encodes an assumption about which fits generalise.

Regularised ERM

$$\text{regularised ERM} \quad \hat{\theta}_\lambda = \arg \min_{\theta \in \Theta} [\hat{R}_{\text{train}}(\theta) + \lambda \Omega(\theta)]$$

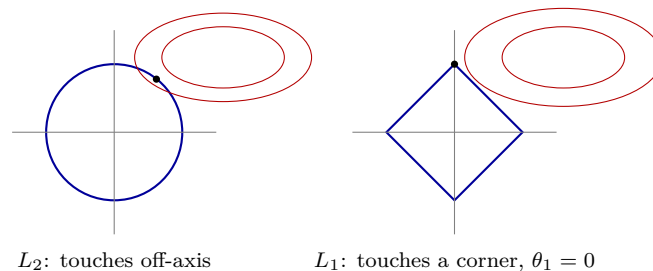
$$L_2 \text{ penalty (ridge)} \quad \Omega_2(\theta) = \|\theta\|_2^2 = \sum_j \theta_j^2$$

$$L_1 \text{ penalty (lasso)} \quad \Omega_1(\theta) = \|\theta\|_1 = \sum_j |\theta_j|$$

Ω is the penalty (it says which parameter values are preferred) and $\lambda \geq 0$ is its strength, a hyperparameter.

Why a penalty reduces capacity without removing parameters (Review Q11). Define the constrained sets $\Theta_c = \{\theta : \Omega(\theta) \leq c\}$ and $\mathcal{H}_c = \{f_\theta : \theta \in \Theta_c\}$. If $c_1 < c_2$ then $\Theta_{c_1} \subseteq \Theta_{c_2}$, hence $\mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$: a smaller budget is a smaller nested family. A larger λ corresponds to a smaller c (the exact map depends on the problem), so a stronger penalty confines the solution to a smaller effective family while the parameter count is untouched. λ is a hyperparameter chosen on validation data: $\hat{\lambda} \in \arg \min_\lambda \hat{R}_{\text{val}}(\hat{f}_\lambda)$.

L_2 versus L_1 (Review Q12). The L_2 ball is round, so the constrained optimum generically touches it at a point with all coordinates nonzero: coefficients shrink smoothly and all variables stay. The L_1 ball is a diamond with corners *on the axes*; the level sets of the training risk usually first touch the diamond at a corner, where some coordinates are exactly zero: embedded feature selection, though correlated features make the selected subset unstable.



Early stopping (Review Q13). Iterative optimisation produces $\hat{\theta}_0, \hat{\theta}_1, \dots$; early iterations learn recurring patterns, late ones fit exceptions and noise. Keep the checkpoint with the smallest validation risk:

$$t^* = \arg \min_t \hat{R}_{\text{val}}(\hat{\theta}_t), \quad \hat{\theta}_{ES} = \hat{\theta}_{t^*}.$$

Training time controls effective capacity at a fixed parameter count: implicit regularisation. Validation loss is noisy, so a **patience** p lets training continue p epochs without improvement before stopping, retaining the best checkpoint; too short stops on a fluctuation or misses an epoch-wise second descent, longer costs computation.

3.5 Feature selection and model selection

Feature selection changes the representation: it removes irrelevant variables (which raise estimation difficulty), reduces redundancy, aids interpretation, and drops variables unavailable at decision time. It is a data-dependent choice and must be validated like any other. Exhaustive search over the 2^d subsets, $S^* \in \arg \min_S \hat{R}_{\text{val}}(\hat{f}_S)$, is combinatorial and infeasible ($d = 30$ exceeds a billion). Alternatives: a **correlation filter** $s_j = |\text{Corr}(X_j, Y)|$ (fast, model-free; misses nonlinear relations and interactions; ignores redundancy, two top-ranked features may carry the same information; weak marginal correlation does not prove uselessness jointly); **forward selection** (start empty, add the feature with the best validation gain) and **backward elimination** (start full, remove the least useful), both greedy, an early choice can block a better combination, and backward needs the full model fitted. Every data-dependent scoring belongs inside the training pipeline and, in cross-validation, must be redone inside each training fold.

Generalising from λ to a whole configuration h (features, hyperparameters, thresholds), the selection reads $\hat{h} \in \arg \min_{h \in \mathcal{C}} \hat{R}_{\text{val}}(\hat{f}_h)$, and in K -fold form

$$\hat{R}_{CV}(h) = \frac{1}{K} \sum_{j=1}^K \hat{R}_{\text{val}}^{(j)}(\hat{f}_h^{(-j)}), \quad \hat{h} \in \arg \min_h \hat{R}_{CV}(h),$$

costing about K fits per configuration and assuming exchangeable observations.

Mastery: the winner's curse of validation (Tutorial Q5d, Review Q15)

Two configurations have the same population error R ; their validation errors are $\hat{R}_i = R + \varepsilon_i$ with independent $\varepsilon_i \sim N(0, \sigma^2)$. We select the smaller. Using $\min(a, b) = \frac{a+b-|a-b|}{2}$ and $\mathbb{E}|\varepsilon_1 - \varepsilon_2| = 2\sigma/\sqrt{\pi}$ (the difference is $N(0, 2\sigma^2)$),

$$\mathbb{E}[\min(\hat{R}_1, \hat{R}_2)] = R + \frac{\mathbb{E}\varepsilon_1 + \mathbb{E}\varepsilon_2}{2} - \frac{\mathbb{E}|\varepsilon_1 - \varepsilon_2|}{2} = R - \frac{\sigma}{\sqrt{\pi}}.$$

With $R = 0.20$, $\sigma = 0.02$: $0.20 - 0.0113 = 0.189$. The reported validation error is optimistic by $\sigma/\sqrt{\pi}$ even though neither model is better; with many configurations the minimum of many noisy estimates is more optimistic still. This is why the selected configuration must be re-measured on an untouched test set.

3.6 Reliable evaluation: splits, metrics, leakage

The split must reproduce the deployment novelty. **Random split**: only when new cases are like independent draws from the same population (goal (i) of Tutorial Q4). **Stratified**: random but preserving class proportions (stabilises rare-class counts; cannot override dependence). **Grouped**: every device, user, patient, organisation or *attack episode* entirely in one subset, so performance is measured on unseen groups (goal (iii)). **Temporal**: train on the past, evaluate on later data; preserves the direction of prediction and reveals drift (goal (ii)). Nearly duplicate flows from one episode scattered across train and test are the textbook failure.

Metrics for imbalanced classes (positive = attack)

TP : attacks flagged; FN : attacks missed; FP : benign flows flagged; TN : benign flows left alone.

$$\begin{aligned}
 \text{accuracy} & \frac{TP + TN}{TP + FP + TN + FN} \\
 \text{recall (sensitivity, true-positive rate)} & \frac{TP}{TP + FN} \\
 \text{precision} & \frac{TP}{TP + FP} \\
 \text{specificity (true-negative rate)} & \frac{TN}{TN + FP} \\
 \text{false-positive rate} & \frac{FP}{FP + TN} = 1 - \text{specificity} \\
 \text{balanced accuracy} & \frac{\text{recall} + \text{specificity}}{2} \\
 F_1 \text{ score} & \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}
 \end{aligned}$$

Recall: of the real attacks, how many did we catch? Precision: of our alarms, how many were real? Specificity: of the benign flows, how many were left alone?

Worked numbers: Tutorial Q4c

10,000 flows, 100 attacks; $TP = 70$, $FP = 180$, $FN = 30$, $TN = 9720$. Accuracy = $9790/10000 = 0.979$; precision = $70/250 = 0.28$; recall = $70/100 = 0.70$; $F_1 = 2(0.28)(0.70)/0.98 = 0.40$. “Always benign” scores accuracy 0.99, higher, with recall 0 and undefined precision. Accuracy hides complete minority failure. F_1 is high only when both precision and recall are, ignores true negatives, and weighs the two errors symmetrically; the metric and the threshold must follow the costs of missed attacks, false alarms and alert volume.

Leakage: development uses information unavailable at deployment-time decisions. Forms: duplicates or near-duplicates across subsets; the same device, session or patient in training and test; predictors created after the outcome or after a human investigation (target leakage); preprocessing fitted on validation or test data; hyperparameters tuned on test results. It inflates the estimate. Preprocessing is part of the fitted pipeline (Review Q19): standardisation estimates

$$\hat{\mu}_j = \frac{1}{N_{tr}} \sum_{i \in tr} x_j^{(i)}, \quad \hat{\sigma}_j^2 = \frac{1}{N_{tr}} \sum_{i \in tr} (x_j^{(i)} - \hat{\mu}_j)^2$$

on the training set only and reuses them unchanged on validation, test and deployment data; the same for imputation, feature selection, dimensionality reduction and resampling, per fold in cross-validation.

Worked numbers: audit of a misleading experiment (Tutorial Q4a, Exercise 10)

The team standardises before splitting (preprocessing leakage), splits rows at random although episodes produce near-duplicates (group leakage, test contains copies of training), tries 100 or 500 configurations and picks the best *test* accuracy (test used as validation, winner’s curse), reports that number as final (no untouched set), uses accuracy on a 1% positive class (hides minority failure), and thresholds at 0.5 although a missed attack costs 100 times a false alarm (wrong decision loss). Redesign: group by episode and, for drift, split by time; fit scaler, imputer and feature selector inside each training fold; choose the configuration and the threshold on validation; report test recall, precision, false-positive rate and alarms per 100,000 benign flows; keep training loss (cross-entropy), validation criterion (cost-weighted), test metric and deployment decision loss as four distinct objects.

Diagnosis with learning curves (error versus training-set size, both curves):

Pattern	Likely issue	Response
both errors high	underfitting	better representation or more capacity
training low, validation high	high variance	regularise, reduce capacity, more representative data
errors close, both falling with N	data-limited	more representative examples will help
test degrades over time	distribution shift	refresh data, time-aware evaluation

Curves are evidence, not proofs; read them with domain knowledge and error analysis.

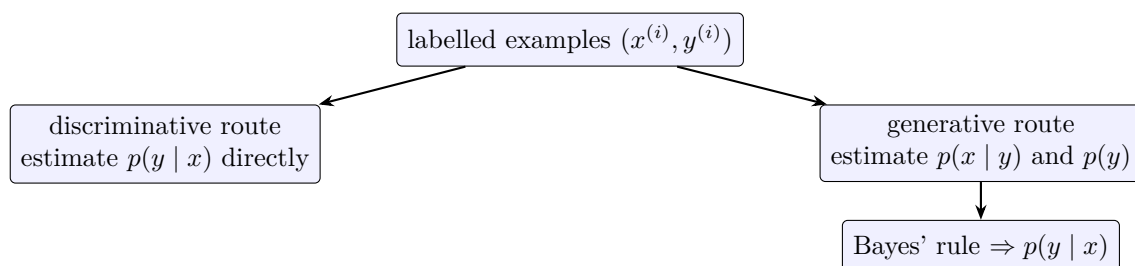
The thread so far

Training loss is a fit, not a generalisation estimate. Inductive bias (data, representation, $\mathcal{H}$, regularisation, optimiser) is unavoidable. Capacity trades approximation for estimation error; bias is systematic across samples, variance is spread across samples, noise is irreducible. Control capacity by family, penalty (L_2 shrinks, L_1 zeroes), early stopping, feature selection. Validation chooses and is itself optimistic by about $\sigma/\sqrt{\pi}$; test measures once; the split mirrors deployment; nothing leaks.

4 Discriminative and generative classification (Lecture 4)

4.1 The decision is easy; the posterior is the problem

With $Y \in \{1, \dots, C\}$ and $X \in \mathbb{R}^d$, if the posterior $p(y = c | x)$ were known the Bayes rule would decide, and under zero-one loss $f^*(x) = \arg \max_c p(y = c | x)$. The learning problem is that the posterior must be estimated from $\mathcal{D}_N$. Two routes:



A discriminative model asks *where the label changes* as a function of the features. A generative classifier builds a probabilistic description of each class and compares how well each explains the input. Neither is universally better; it depends on data amount, credibility of assumptions, representation and objective. “Generative” names the modelled direction, not the task: a class-conditional model can classify an input or, in principle, generate inputs of a chosen class, which links these classifiers to generative AI. Naive Bayes remains a cheap spam baseline; class-conditional models score how compatible an event is with normal or known-malicious behaviour.

4.2 The discriminative route

Scores $\rightarrow$ probabilities $\rightarrow$ action. The model first outputs a real-valued **score** (logit) per class, then a probability, then a decision: $\{s_{\hat{\theta},c}(x)\} \rightarrow \{p_{\hat{\theta}}(y = c | x)\} \rightarrow \hat{y}(x)$. The score family fixes how inputs are ranked and the geometry of the boundary; the probability transform quantifies relative uncertainty; the decision loss fixes the action.

Scores, sigmoid and softmax

score (logit)	$s_{\hat{\theta}}(x) \in \mathbb{R}$ one per class (a single one in the binary case)
sigmoid (logistic function)	$\sigma(s) = \frac{1}{1 + e^{-s}}$
binary posterior estimate	$p_{\hat{\theta}}(y = 1 x) = \sigma(s_{\hat{\theta}}(x))$
binary decision, zero-one loss	$\hat{y}(x) = 1 \iff p_{\hat{\theta}}(y = 1 x) > \frac{1}{2} \iff s_{\hat{\theta}}(x) > 0$
softmax (multiclass posterior)	$p_{\theta}(y = c x) = \frac{\exp s_{\theta,c}(x)}{\sum_{k=1}^C \exp s_{\theta,k}(x)}$

In words: the sigmoid squeezes any real score into $(0, 1)$; softmax makes C scores positive and rescales them so they sum to 1.

Mastery: softmax is shift-invariant

Add the same constant a to every score: $\frac{e^{s_c+a}}{\sum_k e^{s_k+a}} = \frac{e^a e^{s_c}}{e^a \sum_k e^{s_k}} = \frac{e^{s_c}}{\sum_k e^{s_k}}$. So only the $C - 1$ differences between scores are identifiable; C scores are kept for symmetry and a simple output layer. For $C = 2$ softmax reduces to the sigmoid of the score difference.

Logistic regression uses $s_{\hat{\theta}}(x) = \hat{w}^\top x + \hat{b}$, so the 0.5 contour is the hyperplane $\hat{w}^\top x + \hat{b} = 0$; a nonlinear score $g(x)$ gives the boundary $g(x) = 0$. The threshold does not make geometry linear, the score family does. A cost threshold $\tau = C_{FP}/(C_{FP} + C_{FN})$ moves the boundary to $\sigma(s) = \tau$, i.e. $s = \log \frac{\tau}{1-\tau}$, a *parallel* hyperplane.

Worked numbers: Tutorial Q6

$s = 1.2$: $\sigma(1.2) = 1/(1 + e^{-1.2}) = 0.769$; true label 1, cross-entropy $-\ln 0.769 = 0.263$. Costs $C_{FP} = 4$, $C_{FN} = 16$: threshold $4/20 = 0.2$; $0.769 > 0.2$, declare attack. Two-feature model $s = -3 + 2x_1 + x_2$: boundary at threshold 0.5 is $2x_1 + x_2 = 3$; at threshold 0.2, $s = \ln(0.2/0.8) = -1.386$, boundary $2x_1 + x_2 = 1.614$, parallel and shifted toward the origin, so the attack region (above the line) grows: costlier misses mean alarming on weaker evidence.

Mastery: cross-entropy is the negative conditional log-likelihood

Per example, $\ell_{CE}^{(i)}(\theta) = -\sum_{c=1}^C \mathbf{1}(y^{(i)} = c) \log p_{\theta}(y = c | x^{(i)}) = -\log p_{\theta}(y^{(i)} | x^{(i)})$. If the labels are conditionally independent given their inputs (i.i.d. sampling), the conditional likelihood of the observed labels factorises,

$$\begin{aligned} \text{conditional likelihood} \quad L_{\text{cond}}(\theta) &= \prod_{i=1}^N p_{\theta}(y^{(i)} | x^{(i)}) \\ \text{negative conditional log-likelihood} \quad -\log L_{\text{cond}}(\theta) &= -\sum_{i=1}^N \log p_{\theta}(y^{(i)} | x^{(i)}) = N \hat{R}_{CE}(\theta) \end{aligned}$$

Minimising the empirical cross-entropy is maximising the conditional likelihood; the factor N and the sign do not change the minimiser.

Deep discriminative classifiers learn a chain of representations and end in scores:

$$h^{(0)} = x, \quad h^{(\ell)} = \varphi(W_{\ell} h^{(\ell-1)} + b_{\ell}), \quad s_{\hat{\theta}}(x) = W_L h^{(L-1)} + b_L, \quad p_{\hat{\theta}}(y = c | x) = \text{softmax}(s_{\hat{\theta}}(x))_c,$$

trained end to end with cross-entropy; the representation is optimised for predicting the label, not for explaining the input.

What the discriminative objective does not give. It need not learn whether x is plausible;

a large maximum posterior does not mean the input resembles the training data (softmax is a *relative* comparison among known classes, so an unfamiliar input can still get a posterior near one); it gives no mechanism to sample inputs or marginalise missing features. **Class imbalance** is a separate matter: the fitted posterior reflects the training frequencies, and the real difficulty is that few minority examples cannot pin down the boundary; weighting or resampling help in that regime and may otherwise distort calibration (Tutorial Q6g).

4.3 The generative route

Generative story, joint, posterior

generative story	$Y \sim \text{Categorical}(\pi), \quad X \mid (Y = c) \sim p_\theta(\cdot \mid y = c)$
class prior	$\pi_c = p(y = c)$
class-conditional model	$p_\theta(x \mid y = c)$
joint model	$p_\phi(x, y = c) = \pi_c p_\theta(x \mid y = c), \quad \phi = (\pi, \theta)$
posterior by Bayes' rule	$p_\phi(y = c \mid x) = \frac{\pi_c p_\theta(x \mid y = c)}{\sum_{k=1}^C \pi_k p_\theta(x \mid y = k)}$

In words: nature first draws a class with probability π_c , then draws the features from that class's distribution; to classify, reverse this with Bayes' rule.

Classification compares prior-weighted likelihoods; a rare class wins if it explains the input much better. Example: benign $\pi = 0.99$, $p(x \mid \text{benign}) = 0.001$; attack $\pi = 0.01$, $p(x \mid \text{attack}) = 0.20$. Products 0.00099 and 0.002, so $p(\text{attack} \mid x) = 0.002/0.00299 \approx 0.67$.

Generative maximum likelihood

joint likelihood	$L(\phi) = \prod_{i=1}^N p_\phi(x^{(i)}, y^{(i)})$
negative log-likelihood	$-\log L(\phi) = -\sum_{i=1}^N \left[\log \pi_{y^{(i)}} + \log p_\theta(x^{(i)} \mid y^{(i)}) \right]$

Each example must be explained in both its class and its features, a stronger demand than explaining the class given the features. Two separate approximations: **shape approximation** (the family $p_\theta(x \mid y)$ may not contain the truth) and **finite-sample estimation** ($\hat{\phi}$ differs from the best member of the family). Both move the fitted posterior away from the true one.

4.4 Gaussian class models: LDA and QDA

For continuous features, $p_\theta(x \mid y = c) = \mathcal{N}(x; \mu_c, \Sigma_c)$: μ_c sets the centre, Σ_c the scale, orientation and feature dependence (spherical gives circular contours, correlation rotates and elongates them).

Gaussian class model and its maximum-likelihood estimates

With N_c examples in class c ,

$$\text{multivariate Gaussian density} \quad \mathcal{N}(x; \mu_c, \Sigma_c) = \frac{1}{(2\pi)^{d/2} |\Sigma_c|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_c)^\top \Sigma_c^{-1} (x - \mu_c) \right)$$

$$\text{prior estimate} \quad \hat{\pi}_c = \frac{N_c}{N}$$

$$\text{class mean} \quad \hat{\mu}_c = \frac{1}{N_c} \sum_{i: y^{(i)}=c} x^{(i)}$$

$$\text{class covariance (ML)} \quad \hat{\Sigma}_c = \frac{1}{N_c} \sum_{i: y^{(i)}=c} (x^{(i)} - \hat{\mu}_c)(x^{(i)} - \hat{\mu}_c)^\top$$

The denominator is N_c (maximum likelihood), not the $N_c - 1$ of the unbiased sample covariance. No iteration needed.

Worked numbers: one-dimensional check

Class 0 = {0, 1, 2, 0, 1, 2}, class 1 = {3, 4, 5}: $\hat{\pi} = (2/3, 1/3)$, $\hat{\mu} = (1, 4)$, $\hat{\sigma}_0^2 = \hat{\sigma}_1^2 = 2/3$. At $x = 2$: $\mathcal{N}(2; 1, 2/3) = \frac{1}{\sqrt{2\pi \cdot 2/3}} e^{-1/(4/3)} = 0.4886 \times 0.472 = 0.231$, $\mathcal{N}(2; 4, 2/3) = 0.4886 \times e^{-3} = 0.0243$.

Weighted: $\frac{2}{3}(0.231) = 0.154$ and $\frac{1}{3}(0.0243) = 0.0081$, so $p(y = 0 \mid 2) = 0.154/0.162 \approx 0.950$: distance to the centre and the larger prior both count.

Mastery: from the Gaussian log-density to the LDA score (Tutorial Q7 Part I)

Define the generative score $s_{\hat{\phi},c}(x) = \log \hat{\pi}_c + \log p_{\hat{\theta}}(x \mid y = c)$. Then $\log p_{\hat{\phi}}(y = c \mid x) = s_{\hat{\phi},c}(x) - \log \sum_k \exp s_{\hat{\phi},k}(x)$, and the second term is the same for every class, so the decision can be made on the scores alone. The Gaussian log-density is

$$\log \mathcal{N}(x; \mu_c, \Sigma_c) = -\frac{d}{2} \log 2\pi - \frac{1}{2} \log |\Sigma_c| - \frac{1}{2} (x - \mu_c)^\top \Sigma_c^{-1} (x - \mu_c).$$

Expand the quadratic: $(x - \mu_c)^\top \Sigma_c^{-1} (x - \mu_c) = x^\top \Sigma_c^{-1} x - 2x^\top \Sigma_c^{-1} \mu_c + \mu_c^\top \Sigma_c^{-1} \mu_c$.

LDA: all classes share Σ . Then $-\frac{d}{2} \log 2\pi$, $-\frac{1}{2} \log |\Sigma|$ and $-\frac{1}{2} x^\top \Sigma^{-1} x$ are identical for every class and drop out, leaving

$$s_{\hat{\phi},c}(x) = x^\top \hat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^\top \hat{\Sigma}^{-1} \hat{\mu}_c + \log \hat{\pi}_c,$$

which is *linear* in x . The boundary between classes c and k is $s_c(x) = s_k(x)$, i.e. $x^\top \hat{\Sigma}^{-1} (\hat{\mu}_c - \hat{\mu}_k) = \text{const}$: a hyperplane. “Linear” names the boundary, not the density.

QDA: class-specific Σ_c . Only the 2π term is common, so

$$s_{\hat{\phi},c}(x) = -\frac{1}{2} (x - \hat{\mu}_c)^\top \hat{\Sigma}_c^{-1} (x - \hat{\mu}_c) - \frac{1}{2} \log |\hat{\Sigma}_c| + \log \hat{\pi}_c,$$

a squared Mahalanobis distance to the centre, a term for how diffuse the class is, and the prior; the class-specific quadratic terms give curved boundaries.

Worked numbers: Tutorial Q7 Part I with numbers

$\mu_0 = (0, 0)$, $\mu_1 = (2, 2)$, $\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$, $\pi_0 = 0.75$, $\pi_1 = 0.25$. Inverse: $\Sigma^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}$ (check: the product is $\frac{1}{1-\rho^2} \begin{pmatrix} 1-\rho^2 & 0 \\ 0 & 1-\rho^2 \end{pmatrix} = I$). With $\rho = 0.5$, $\Sigma^{-1} = \begin{pmatrix} 4/3 & -2/3 \\ -2/3 & 4/3 \end{pmatrix}$, $\Sigma^{-1} \mu_1 = (4/3, 4/3)$, $\mu_1^\top \Sigma^{-1} \mu_1 = 16/3$. Scores: $s_0(x) = \ln 0.75$; $s_1(x) = \frac{4}{3}(x_1 + x_2) - \frac{8}{3} + \ln 0.25$. Boundary $s_0 = s_1$: $\frac{4}{3}(x_1 + x_2) = \frac{8}{3} + \ln 3$, i.e. $x_1 + x_2 = 2 + \frac{3}{4} \ln 3 \approx 2.82$, a line perpendicular to the direction $\Sigma^{-1}(\mu_1 - \mu_0)$. Test point (1.25, 1.25): sum $2.5 < 2.82$, class 0; $s_1 - s_0 = \frac{4}{3}(2.5) - \frac{8}{3} - \ln 3 = -0.432$, so $p(y = 1 \mid x) = \sigma(-0.432) = 0.39$. Equal priors: boundary $x_1 + x_2 = 2$, the midpoint line; now $s_1 - s_0 = 0.667$, $p(y = 1 \mid x) = 0.66$, class 1. The prior shifts the line by $\frac{3}{4} \ln 3$ toward the rarer

class while the class models are untouched.

Principle: assumptions about the class-conditionals determine the boundary geometry. LDA shares structure and estimates fewer parameters; QDA is more flexible and needs more data. A $d \times d$ covariance has $d(d+1)/2$ free parameters: 55 at $d = 10$, 5050 per class at $d = 100$; with few observations it is unstable or singular. **Covariance shrinkage** $\hat{\Sigma}_\lambda = (1-\lambda)\hat{\Sigma} + \lambda\alpha I$, $0 \leq \lambda \leq 1$ chosen on validation, interpolates between the sample covariance and a scaled identity (no correlations, one reference variance). Pushing this simplification to the limit gives Naive Bayes.

4.5 Naive Bayes

Conditional independence and the additive score

$$\text{Naive Bayes assumption} \quad p_\theta(x \mid y = c) = \prod_{j=1}^d p_\theta(x_j \mid y = c)$$

$$\text{Naive Bayes class score} \quad s_{\hat{\phi},c}(x) = \log \hat{\pi}_c + \sum_{j=1}^d \log p_{\hat{\theta}}(x_j \mid y = c)$$

In words: once the class is known, the features are treated as independent; the score adds one log-likelihood term per feature to the log prior, and the class with the largest score is predicted.

Each feature is a separate clue once the class is known. The assumption is strong (flow duration, packet counts and byte statistics stay correlated within an attack class) but it turns one hard d -dimensional density estimate into d one-dimensional ones, which are stable with little data. A misspecified density can still rank classes correctly: exact densities are sufficient, not necessary, for the right argmax. The risk: correlated features *repeat the same evidence*: if two features are copies, their log-likelihood ratio is counted twice and the posterior is pushed too far toward one class; the classifier may still classify well while its probabilities are too extreme, and removing redundant features improves calibration. Each one-feature model can be whatever fits the feature's support and shape (Gamma or log-normal for positive skewed, Student t for heavy tails, Poisson or negative binomial for counts, histograms or kernel density estimates with enough data), validated like any choice; $\log(1+x)$ often makes a skewed positive feature near-Gaussian, while centring and scaling do not change its shape.

Worked numbers: Tutorial Q7 Part II

$x_1 = 1$ (port 22), $x_2 = 1$ (short duration). Attack: $0.10 \times 0.60 \times 0.70 = 0.042$. Benign: $0.90 \times 0.05 \times 0.20 = 0.009$. $p(\text{attack} \mid x) = 0.042/0.051 = 0.82$. The two clues were multiplied as if independent given the class; if short durations on port 22 are the same phenomenon, the evidence was double-counted.

Variants. Gaussian (continuous measurements), Bernoulli (binary indicators), Multinomial (token or event counts), Hybrid (a distribution per feature type for mixed data; contributions multiply under one class prior, which appears only once).

Gaussian NB and variance smoothing. $x_j \mid (y = c) \sim \mathcal{N}(\mu_{cj}, \sigma_{cj}^2)$. A zero fitted variance makes the density undefined, a tiny one lets a single feature dominate, so regularise $\tilde{\sigma}_{cj}^2 = \hat{\sigma}_{cj}^2 + \lambda s_j^2$ with s_j^2 the overall training variance of feature j and a small $\lambda > 0$ (validated if it matters).

Multinomial Naive Bayes for text and additive smoothing

Vocabulary of size V ; a document is the count vector $x = (x_1, \dots, x_V)$, x_j = number of times

word j occurs. n_{cj} = training count of token j in class c ; $\alpha > 0$ = smoothing strength.

token probability in class c $\theta_{cj} = p_{\theta}(\text{token} = j \mid y = c), \quad \sum_j \theta_{cj} = 1$

document likelihood $p_{\theta}(x \mid y = c) \propto \prod_{j=1}^V \theta_{cj}^{x_j}$

smoothed token estimate $\hat{\theta}_{cj} = \frac{n_{cj} + \alpha}{\sum_{\ell=1}^V n_{c\ell} + \alpha V}$

smoothed categorical estimate $p_{\hat{\theta}}(x_j = v \mid y = c) = \frac{n_{c_j v} + \alpha}{n_{cj} + \alpha K_j}$

$\propto$ because the multinomial coefficient, common to all classes, is dropped; $0^0 = 1$; counts are kept, word order is ignored. For a categorical feature with K_j levels, $n_{c_j v}$ counts value v in class c and $n_{cj} = \sum_v n_{c_j v}$. With $\alpha = 1$ this is often called Laplace smoothing.

Worked numbers: why smoothing is not optional

Vocabulary (free, win, prize, meeting, project, report); spam counts (3, 2, 2, 0, 0, 0), ham counts (0, 0, 0, 3, 2, 2), seven tokens each. Unsmoothed, “free meeting” has likelihood $\frac{3}{7} \cdot \frac{0}{7} = 0$ under spam and $\frac{0}{7} \cdot \frac{3}{7} = 0$ under ham: one unseen word zeroes the product and no posterior exists. With $\alpha = 1$, $V = 6$: $\hat{\theta}_{\text{spam}} = \frac{1}{13}(4, 3, 3, 1, 1, 1)$, $\hat{\theta}_{\text{ham}} = \frac{1}{13}(1, 1, 1, 4, 3, 3)$; equal priors give $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{1}{13} = \frac{2}{169}$ for both, so $p(\text{spam} \mid x) = \frac{1}{2}$. Larger α moves estimates toward uniform. A port value never seen in training (Tutorial Q7d) needs a representation: create an “other” level during training, map unseen values to it, and smoothing gives it a nonzero probability.

4.6 Comparing the two routes

	Discriminative	Generative
Modelled quantity	posterior $p_{\theta}(y \mid x)$ or boundary	prior and class model $\pi_c p_{\theta}(x \mid y = c)$
Input assumptions	usually fewer	usually stronger
Primary objective	predict the class	describe class-specific inputs
Missing features	separate treatment	can sometimes be marginalised
Sampling inputs	usually unavailable	possible in principle
Raw high-dimensional input	often practical	full density estimation is hard

Generative structure shares a restricted description of each class and so can reduce estimation error with little labelled data when the assumptions roughly hold; a wrong family creates approximation bias that more data does not remove. A flexible discriminative model can eventually reach lower error because it spends no capacity on aspects of X irrelevant to Y . For the flow application: prefer discriminative when many labelled flows exist and the raw features are hard to model; prefer generative when labels are scarce, a plausible per-class model can be written, or a compatibility check against known classes is wanted.

4.7 Imbalance, prior shift, calibration

Imbalance without shift. If training and deployment share the joint distribution, an imbalanced prior is not an error; the posterior should reflect it. The real problem is few minority examples (small N_1), where weighting or resampling can help estimation; otherwise they may distort probabilities. A classifier is **calibrated** if among cases given probability u about a fraction u are positive. Calibration concerns the meaning of probabilities; the cost threshold concerns the action. Separate questions.

Mastery: class-prior shift and its correction (Tutorial Q6e,f)

Shift: $p_d(x | y = c) = p_t(x | y = c)$ but $\pi_{d,c} \neq \pi_{t,c}$ ($t = \text{training}$, $d = \text{deployment}$).

Generative: keep the fitted class-conditionals, swap the priors: $p_d(y = c | x) = \pi_{d,c} p_{\hat{\theta}}(x | c) / \sum_k \pi_{d,k} p_{\hat{\theta}}(x | k)$. Nothing is refitted.

Binary discriminative: Bayes' rule gives the training posterior odds $\frac{r_t(x)}{1 - r_t(x)} = \frac{\pi_t}{1 - \pi_t} \cdot \frac{p(x | y = 1)}{p(x | y = 0)}$. Under prior shift the likelihood ratio is unchanged, so replacing the prior odds gives $\frac{r_d}{1 - r_d} = \frac{\pi_d/(1 - \pi_d)}{\pi_t/(1 - \pi_t)} \cdot \frac{r_t}{1 - r_t}$; solving for r_d ,

$$r_d(x) = \frac{(\pi_d/\pi_t) r_t(x)}{(\pi_d/\pi_t) r_t(x) + \frac{1 - \pi_d}{1 - \pi_t} [1 - r_t(x)]}.$$

With $\pi_t = 0.20$, $\pi_d = 0.05$, $r_t = 0.60$: ratio 0.25, numerator 0.15, second term $\frac{0.95}{0.80}(0.40) = 0.475$, so $r_d = 0.15/0.625 = 0.24$. The correction needs r_t to be calibrated. It changes the *probability* because the world changed; the cost threshold changes the *action* because the consequences differ. Both can apply to the same flow, in that order.

4.8 Unfamiliar inputs, sequences, and combined systems

A high $\max_c p_{\hat{\theta}}(y = c | x)$ only says one known class outranks the others; it does not say x resembles training data, which matters when a new attack mechanism lies far from every known class. A generative model adds a compatibility check: pick $c^* = \arg \max_c p_{\hat{\phi}}(y = c | x)$, then

$$\hat{y} = \begin{cases} c^* & \text{if } \log p_{\hat{\theta}}(x | y = c^*) \geq \tau_{c^*}, \\ \text{reject} & \text{otherwise,} \end{cases}$$

with τ_c validated on data that reproduce the intended novelty (attack types held out of training). Low likelihood is evidence of incompatibility, not a guarantee; high-dimensional likelihoods behave oddly and a misspecified model can reject valid cases. Modern systems combine the routes: a discriminative model ranks known classes, a generative or anomaly model checks compatibility, and the policy accepts, sends to review, or rejects.

From flows to windows (Tutorial Q8). A per-flow classifier can miss an attack whose individual flows all look ordinary (scores 0.10 to 0.28, none above 0.70). A second detector aggregates recent behaviour from one source: in a 60-second window ending at each completion time, count flows, distinct destination ports, the failure fraction, and the sum of flow scores. In the tutorial the window ending at $t = 44$ s holds flows 1–4: four flows, four ports, failure fraction 3/4, score sum 0.74, so the scan is detected 27 s after it began. A 20-second window never holds four flows (they are 13 s apart) and never alarms: shorter windows react faster but see less. Discounted accumulation $A_i = 0.5A_{i-1} + s_i$ gives 0.10, 0.23, 0.335, 0.4075, 0.484, crossing 0.45 at $t = 57$ s, later; its half-life is a hidden hyperparameter. Everything in the window must be known at the decision time (no future flows, no later ticket), a shared NAT address mixes several devices into one “source” (group by device or session when available), and windows generated from the same episode must stay on one side of every split.

Language models. An autoregressive model is trained generatively, $p_{\theta}(w_{1:T}) = \prod_{t=1}^T p_{\theta}(w_t | w_{<t})$. A sequence-classification head models $p_{\psi}(y | w_{1:T})$ with the discriminative objective $L_{\text{disc}}(\psi) = -\sum_i \log p_{\psi}(y^{(i)} | w_{1:T_i}^{(i)})$, one label per sequence; or a prompt asks the model to generate or score label tokens. Generative pretraining, discriminative downstream objective.

Choosing a model. Can a credible $p_{\theta}(x | y)$ be stated? Then its structure may save labelled data. Is the goal accurate prediction with many labels? A flexible discriminative model avoids needless

input assumptions. Is the raw input complex and high-dimensional? Learn a representation first, then a simple head. Labels like “generative” never replace deployment-like validation on the same fixed split and rows for every model.

The thread so far

Posterior + decision loss $\Rightarrow$ decision. Discriminative: scores $\rightarrow$ sigmoid/softmax $\rightarrow$ cross-entropy = negative conditional log-likelihood; boundary geometry from the score family; a cost threshold shifts the boundary in parallel. Generative: prior $\times$ class-conditional, Bayes’ rule, joint likelihood; Gaussian models give LDA (shared Σ , linear boundary) and QDA (own Σ_c , quadratic); Naive Bayes factorises features and needs variance or additive smoothing. Imbalance is an estimation problem, prior shift is a posterior correction, calibration is about probability meaning, confidence is not familiarity, and windows need leakage-safe grouping.

One-glance sheet: every formula

Bayes decision rule	$f^*(x) \in \arg \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$; conditional risk of $a = \mathbb{E}[\ell(Y, a) \mid x]$
Squared / absolute / asymmetric loss	conditional mean / conditional median / quantile at level $c_u/(c_u + c_o)$
Zero-one loss	$f^*(x) = \arg \max_c P(Y = c \mid x)$
Cost threshold	positive iff $p > C_{FP}/(C_{FP} + C_{FN})$; with review at C_R : benign $p \leq C_R/C_{FN}$, attack $p \geq 1 - C_R/C_{FP}$
KNN	$\hat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}$; $\hat{P}_k(c \mid x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c)$; 1-NN boundary = perpendicular bisector
Distances	$d_p(x, z) = (\sum_j x_j - z_j ^p)^{1/p}$; weighted $\sqrt{\sum_j w_j (x_j - z_j)^2}$; standardise $z_j = (x_j - \mu_j)/\sigma_j$
Curse of dimensionality	10^d cells; $M_d \sim \text{Bin}(n, r^d)$, $P(M_d \geq 1) = 1 - (1 - r^d)^n$, $r_d \approx (3/n)^{1/d} \rightarrow 1$
Hypothesis space	$\mathcal{H} = \{f_\theta : \theta \in \Theta\}$; linear $w^\top x + b$; polynomial $\sum_{k \leq p} \theta_k x^k$; simplex Δ^{C-1}
Cross-entropy (log loss)	per example $\ell_{CE} = -\log \hat{p}_{\theta, y}(x)$; training loss $\hat{R}_{CE} = -\frac{1}{N} \sum_i \log \hat{p}_{\theta, y^{(i)}}(x^{(i)})$; binary $\ell_{BCE} = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$
Information theory	entropy $H(p) = -\sum p_c \log p_c$; cross-entropy $H(p, \hat{p}) = H(p) + D_{KL}(p \parallel \hat{p})$; $R_{CE} = H(Y \mid X) + \mathbb{E}_X D_{KL}$; $I(X; Y) = H(Y) - H(Y \mid X)$
Risks, ERM	$R(\theta) = \mathbb{E}[\ell(Y, f_\theta(X))]$; $\hat{R}_{\text{train}} = \frac{1}{N} \sum_i \ell(y^{(i)}, f_\theta(x^{(i)}))$; $\hat{\theta} \in \arg \min \hat{R}_{\text{train}}$
Selection, CV	$\hat{\theta}_\lambda = \arg \min_\theta \hat{R}_{\text{train}}(\theta; \lambda)$; $\hat{\lambda} = \arg \min_\lambda \hat{R}_{\text{val}}(\hat{\theta}_\lambda)$; $\hat{R}_{CV} = \frac{1}{K} \sum_j \hat{R}_{\text{val}}^{(j)}$
Generalisation gap	$G(\hat{f}) = R(\hat{f}) - \hat{R}_{\text{train}}(\hat{f})$
Nested capacity	$\mathcal{H}_m \subseteq \mathcal{H}_{m+1} \Rightarrow \min_{\mathcal{H}_{m+1}} \hat{R}_{\text{train}} \leq \min_{\mathcal{H}_m} \hat{R}_{\text{train}}$
Bias–variance	$\mathbb{E}[(Y - \hat{f}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x)$; Bias = $\bar{f} - f^*$; Var = $\mathbb{E}_{\mathcal{D}}[(\hat{f} - \bar{f})^2]$
Regularised ERM	$\arg \min_\theta [\hat{R}_{\text{train}} + \lambda \Omega(\theta)]$; $\Theta_c = \{\Omega \leq c\}$ nested; $\Omega_2 = \ \theta\ _2^2$ (shrink), $\Omega_1 = \ \theta\ _1$ (sparsify)
Early stopping	$t^* = \arg \min_t \hat{R}_{\text{val}}(\hat{\theta}_t)$ with patience p
Winner's curse	$\mathbb{E} \min(\hat{R}_1, \hat{R}_2) = R - \sigma/\sqrt{\pi}$ for two equal models with $N(0, \sigma^2)$ noise
Feature selection	$s_j = \text{Corr}(X_j, Y) $; 2^d subsets; forward / backward greedy; redo inside every fold
Metrics	Accuracy $\frac{TP+TN}{\text{all}}$, Recall $\frac{TP}{TP+FN}$, Precision $\frac{TP}{TP+FP}$, Specificity $\frac{TN}{TN+FP}$, FPR $\frac{FP}{FP+TN}$, BalAcc, $F_1 = \frac{2PR}{P+R}$
Standardisation (train only)	$\hat{\mu}_j = \frac{1}{N_{tr}} \sum x_j^{(i)}$, $\hat{\sigma}_j^2 = \frac{1}{N_{tr}} \sum (x_j^{(i)} - \hat{\mu}_j)^2$
Sigmoid, softmax	$\sigma(s) = 1/(1 + e^{-s})$, decide 1 iff $s > 0$; threshold $\tau \Leftrightarrow s > \log \frac{\tau}{1-\tau}$; softmax $_c = e^{s_c} / \sum_k e^{s_k}$, shift-invariant
Conditional likelihood	$L_{\text{cond}} = \prod_i p_\theta(y^{(i)} \mid x^{(i)})$; $-\log L_{\text{cond}} = N \hat{R}_{CE}$
Network	$h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$, $s = W_L h^{(L-1)} + b_L$
Generative model	$p_\phi(x, c) = \pi_c p_\theta(x \mid c)$; $p(c \mid x) = \pi_c p(x \mid c) / \sum_k \pi_k p(x \mid k)$; $-\log L = -\sum_i [\log \pi_{y^{(i)}} + \log p_\theta(x^{(i)} \mid y^{(i)})]$
Gaussian density	$\mathcal{N}(x; \mu, \Sigma) = (2\pi)^{-d/2} \Sigma ^{-1/2} \exp(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu))$
Gaussian ML	$\hat{\pi}_c = N_c/N$, $\hat{\mu}_c = \text{class mean}$, $\hat{\Sigma}_c = \frac{1}{N_c} \sum (x - \hat{\mu}_c)(x - \hat{\mu}_c)^\top$
Scores	$s_c = \log \hat{\pi}_c + \log p(x \mid c)$; $\log p(c \mid x) = s_c - \log \sum_k e^{s_k}$; binary $p(1 \mid x) = \sigma(s_1 - s_0)$
LDA	$s_c = x^\top \hat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^\top \hat{\Sigma}^{-1} \hat{\mu}_c + \log \hat{\pi}_c$; boundary $x^\top \Sigma^{-1}(\mu_c - \mu_k) = \text{const}$
QDA	$s_c = -\frac{1}{2}(x - \hat{\mu}_c)^\top \hat{\Sigma}_c^{-1}(x - \hat{\mu}_c) - \frac{1}{2} \log \hat{\Sigma}_c + \log \hat{\pi}_c$
2×2 inverse	$\begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}$; $d(d+1)/2$ covariance parameters; $\hat{\Sigma}_\lambda = (1-\lambda)\hat{\Sigma} + \lambda\alpha I$
Naive Bayes	$p(x \mid c) = \prod_j p(x_j \mid c)$; $s_c = \log \hat{\pi}_c + \sum_j \log p(x_j \mid c)$; $\hat{\sigma}_{cj}^2 = \hat{\sigma}_{cj}^2 + \lambda s_j^2$
Multinomial NB	$p(x \mid c) \propto \prod_j \theta_{cj}^{x_j}$; $\hat{\theta}_{cj} = \frac{n_{cj} + \alpha}{\sum_\ell n_{c\ell} + \alpha V}$; categorical $\frac{n_{cjv} + \alpha}{n_{cj} + \alpha K_j}$
Prior shift	generative: swap π ; discriminative: $r_d = \frac{(\pi_d/\pi_t)r_t}{(\pi_d/\pi_t)r_t + \frac{1-\pi_d}{1-\pi_t}(1-r_t)}$
Reject option	$\hat{y} = c^*$ if $\log p(x \mid c^*) \geq \tau_{c^*}$, else reject
Language models	$p(w_{1:T}) = \prod_t p(w_t \mid w_{<t})$; head $L_{\text{disc}} = -\sum_i \log p_\psi(y^{(i)} \mid w_{1:T_i}^{(i)})$

One-glance sheet: the distinctions

Prediction / decision / action	what the model says / what response is chosen under a criterion / what is executed
Regression / classification	numerical output / categorical output; decided by the output, not the input
Supervised / unsupervised / self-sup. / RL	examples / observations / observations with a constructed signal / interactions
Shortcut / leakage	available at decision time but unstable / not available at decision time at all
ℓ_{train} / ℓ_{dec}	guides estimation / guides the action; may coincide or differ
Parameter / hyperparameter	learned from training data / chosen on validation data; labels are procedure-dependent
Objective / algorithm	which values are preferred / how they are found
Train / validation / test	fit θ and preprocessing / select h / measure once, after freezing
Population / empirical risk	expectation over P (unknown) / sample average (observable, sample-dependent)
Inductive bias / statistical bias	preference among solutions / $\bar{f} - f^*$, systematic error across samples
Bias / variance / noise	average fit misses / fits spread across samples / $\sigma^2(x)$, irreducible
Underfitting / overfitting	both errors high / training low, validation high
Approximation / estimation / optimisation error	$\mathcal{H}$ too poor / data cannot pick the best member / solver fails
L_2 / L_1	shrink all / zero some (corners of the diamond)
Random / stratified / grouped / temporal split	i.i.d. cases / keep class ratios / unseen groups or episodes / future data
Discriminative / generative	$p(y x)$ directly / $p(y)$, $p(x y)$ then Bayes
Shape approximation / finite-sample estimation	wrong family / wrong member of the family
LDA / QDA	shared Σ , linear boundary, fewer parameters / own Σ_c , curved boundary, more data
Imbalance / prior shift / cost threshold	estimation problem / posterior correction / action choice
Calibration / threshold	meaning of probabilities / action taken from them
Confidence / familiarity	max posterior among known classes / compatibility with the training data
